Lecture 3: Dictionaries and Tolerant Retrieval

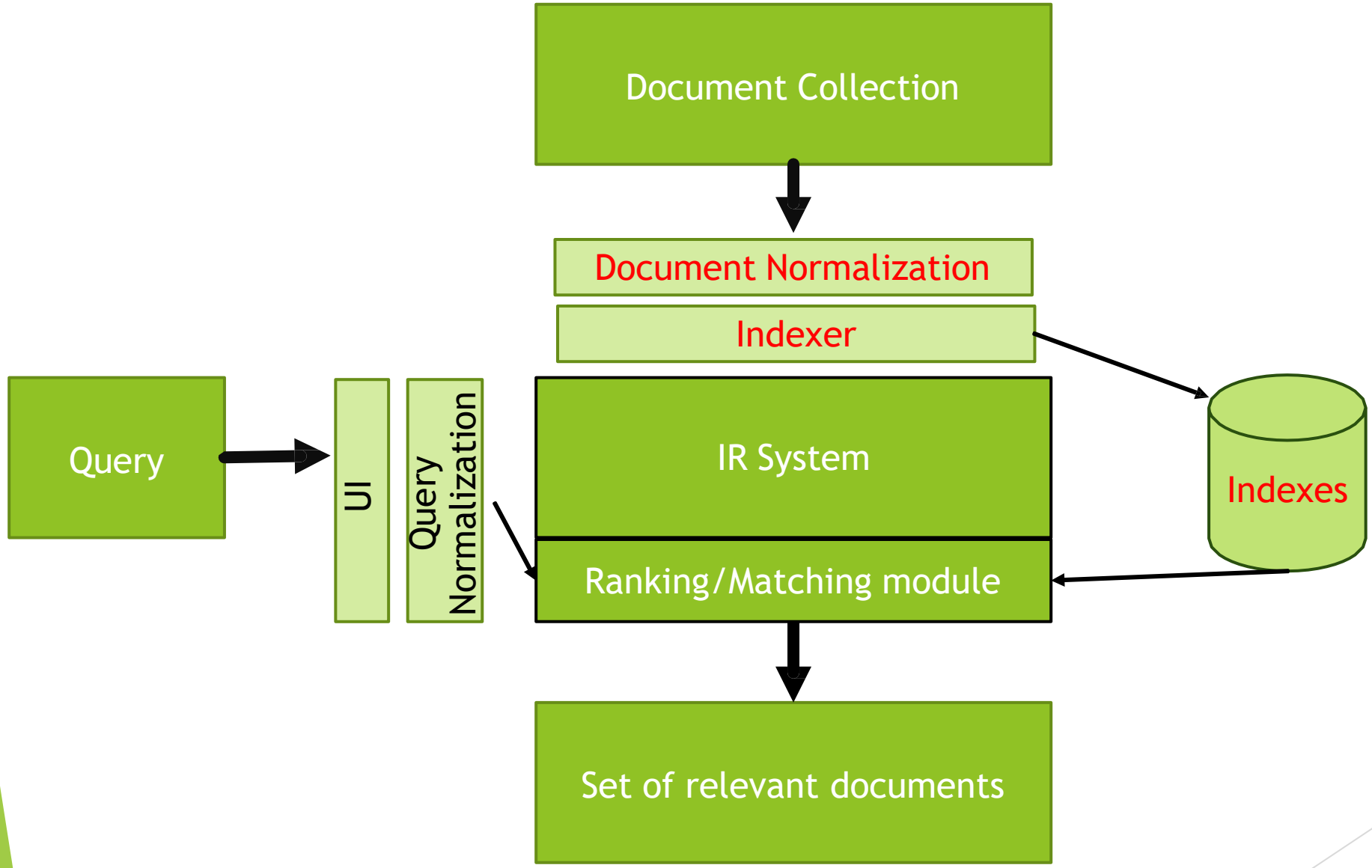
Information Retrieval and Web Analytics

- Recap on the previous lecture

- The type/token distinction:
  - Terms are normalized types put in the dictionary
- Tokenization problems:
  - Hyphens, apostrophes, compounds, CJK
- Term equivalence classing:
  - Numbers, case folding, stemming, lemmatization
- Skip pointers
  - Encoding a tree-like structure in a postings list
- Biword indexes for phrases
- Positional indexes for phrases/proximity queries

This lecture

- Dictionary data structures
- “Tolerant” retrieval
  - Wild-card queries
  - Spelling correction
  - Soundex – words with similar pronunciations


Document Collection
Document Normalization
Indexer
Query
UI
Query Normalization
IR System
Indexes
Ranking/Matching module
Set of relevant documents

Type/token distinction

- Token an instance of a word or term occurring in a document
- Type an equivalence class of tokens

In June, the dog likes to chase the cat in the barn.

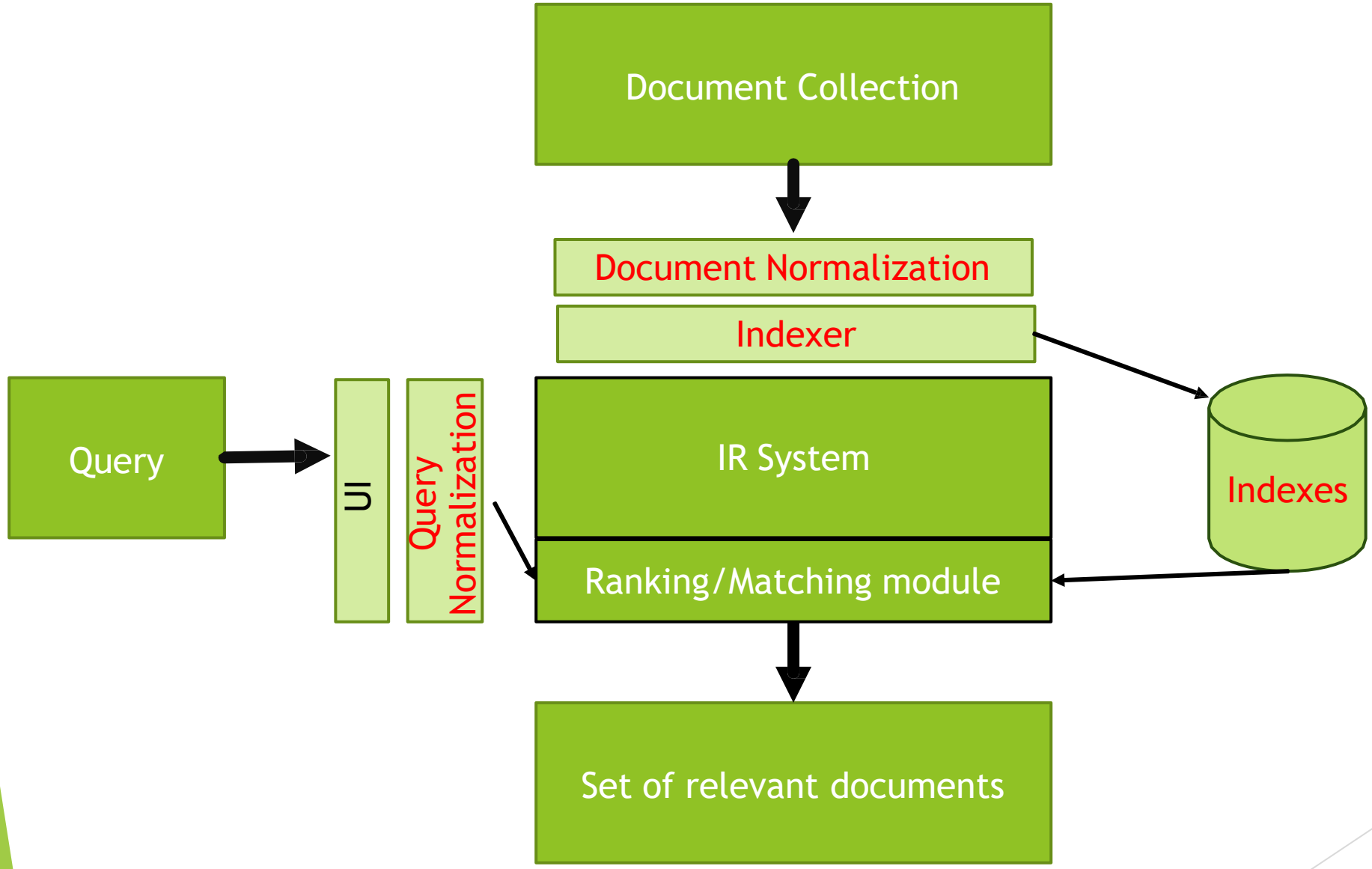
- 12 word tokens
- 9 word types

Problems with equivalence classing

- A term is an equivalence class of tokens.
- How do we define equivalence classes?
- Numbers (3/20/91 vs. 20/3/91)
- Case folding
- Stemming, Porter stemmer
- Morphological analysis: inflectional vs. derivational
- Equivalence classing problems in other languages

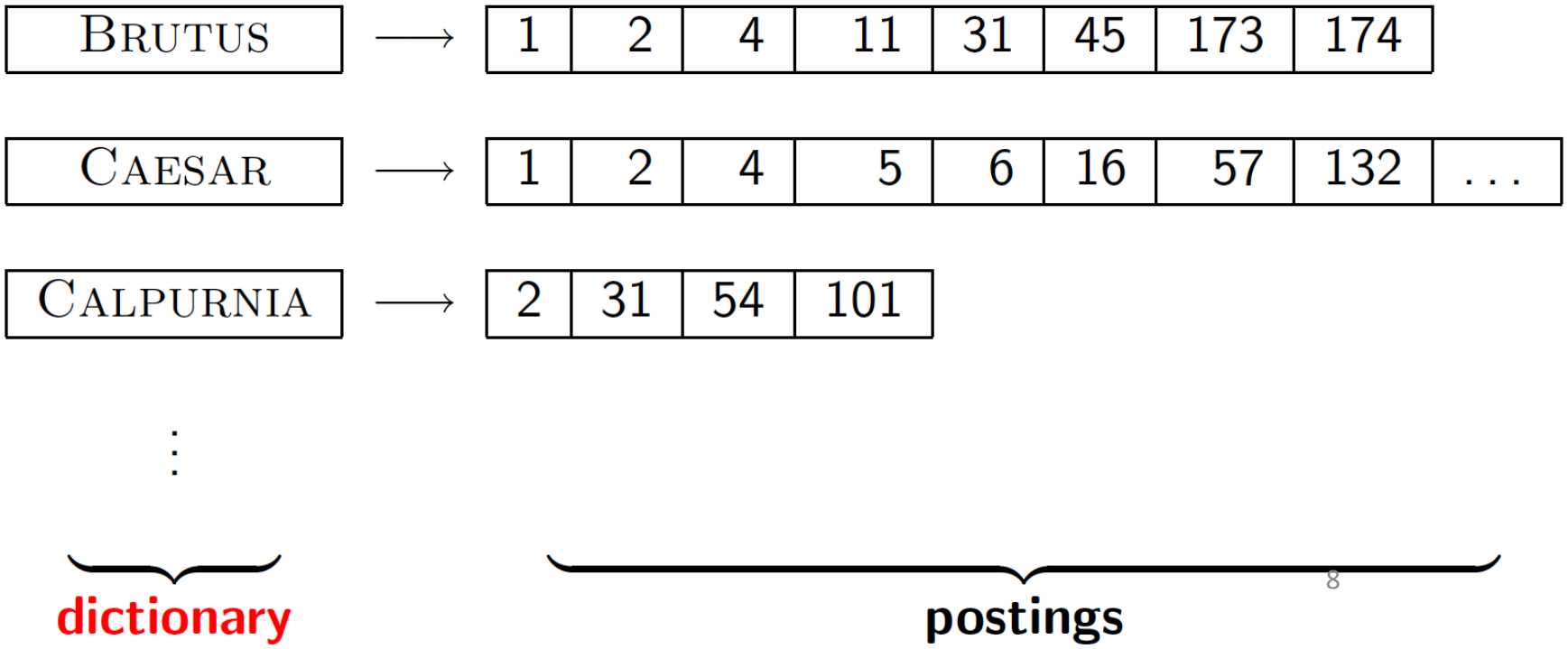
Positional indexes

- Postings lists in a non-positional index: each posting is just a docID
- Postings lists in a positional index: each posting is a docID and a list of positions
- Example query: "to_1 be_2 or_3 not_4 to_5 be_6"
- With a positional index, we can answer
  - phrase queries
  - proximity queries


Document Collection
Document Normalization
Indexer
Query
UI
Query Normalization
IR System
Indexes
Ranking/Matching module
Set of relevant documents

Dictionary data structures for inverted indexes

- The dictionary data structure stores the term vocabulary, document frequency, pointers to each postings list … in what data structure?



A naïve dictionary

- An array of struct:

term	document frequency	pointer to postings list
a	656,265	⟶
aachen	65	⟶
. . .	. . .	. . .
zulu	221	⟶

char[20] int Postings *

20 bytes 4/8 bytes 4/8 bytes

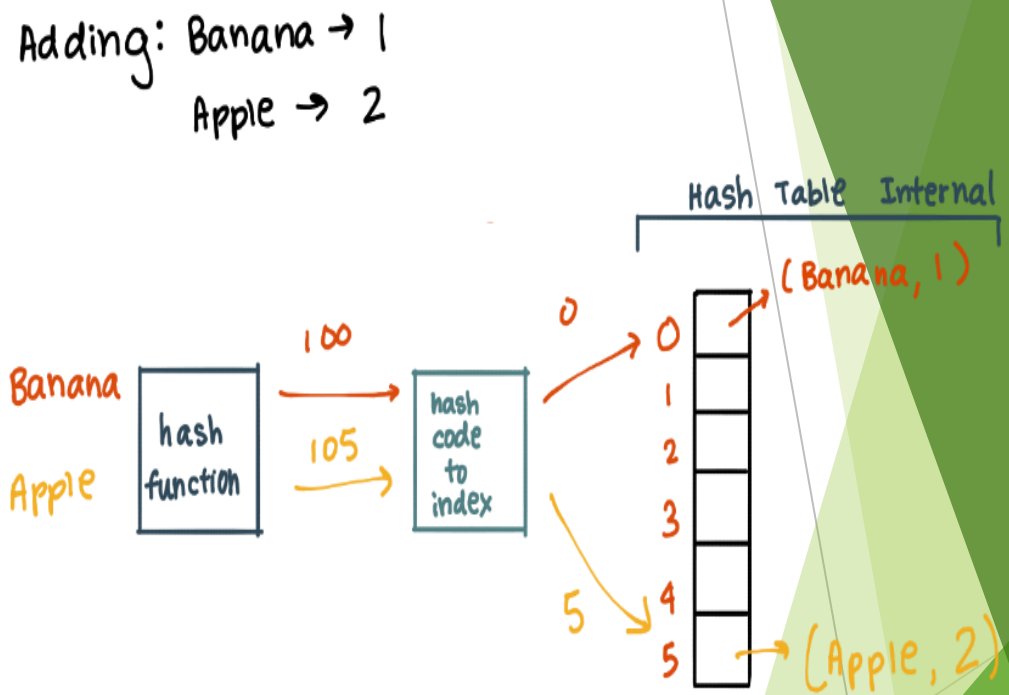
- How do we store a dictionary in memory efficiently?
- How do we quickly look up elements at query time?

Dictionary data structures

- Two main choices:
  - Hash tables
  - Trees
- Some IR systems use hash tables, some trees
- How many keys are we likely to have?
- Is the number likely to remain static, or change a lot – and in the case of changes, are we likely to only have new keys inserted, or to also have some keys in the dictionary be deleted
- What are the relative frequencies with which various keys will be accessed

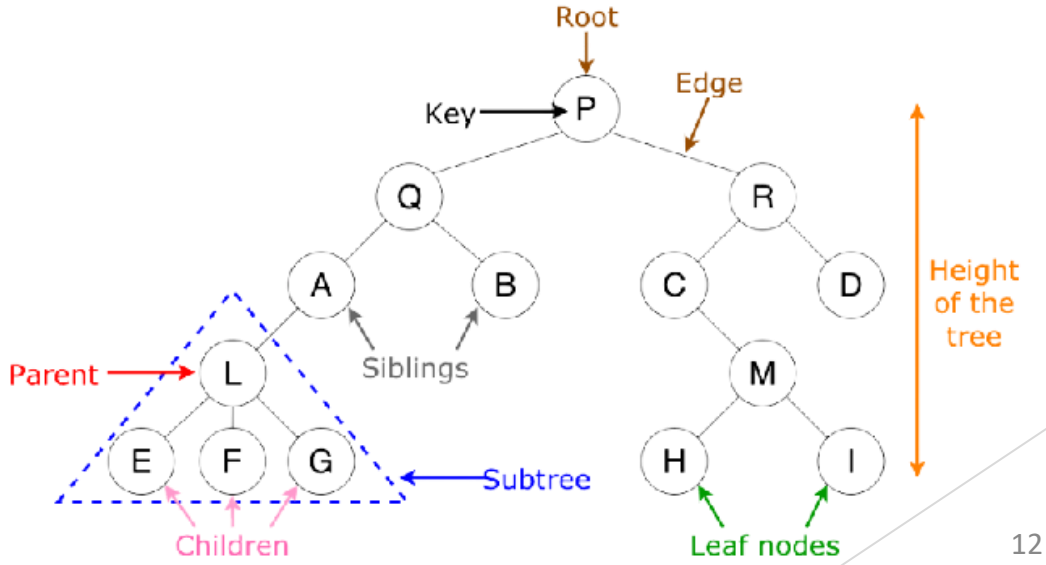
Hashtables

- Each vocabulary term is hashed to an integer
  - (We assume you've seen hashtables before)
- Pros:
  - Lookup is faster than for a tree: O(1)
- Cons:
  - No easy way to find minor variants:
    - judgment/judgement
  - No prefix search [tolerant retrieval]
    - all terms beginning with the prefix automat
  - If vocabulary keeps growing, need to occasionally do the expensive operation of rehashing *everything*

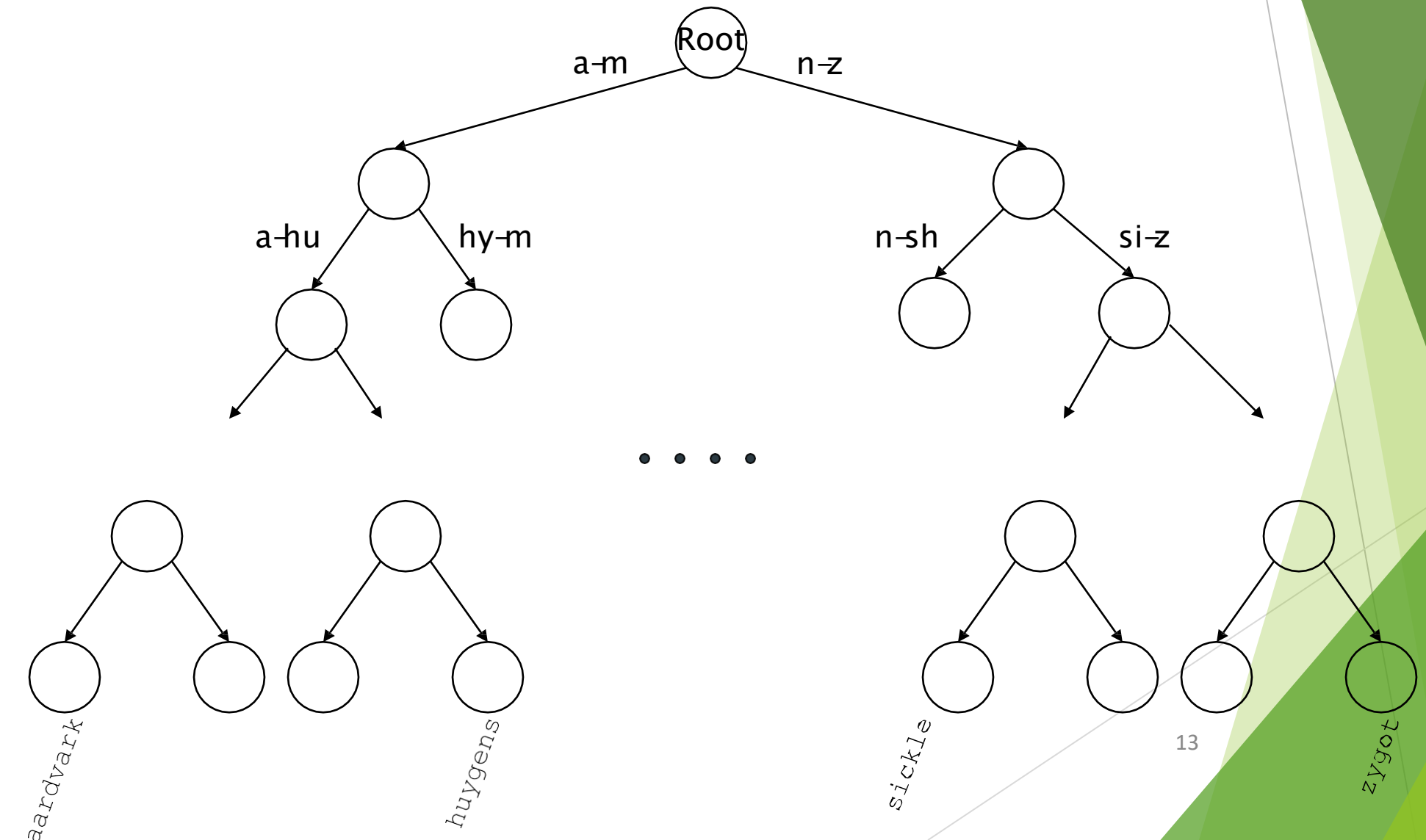


Tree: Binary tree

- Partition vocabulary terms into two subtrees, those whose first letter is between a and m, and the rest (actual terms stored in the leafs).
- Anything that is on the left subtree is smaller than what's on the right. Trees solve the prefix problem (find all terms starting with automat).



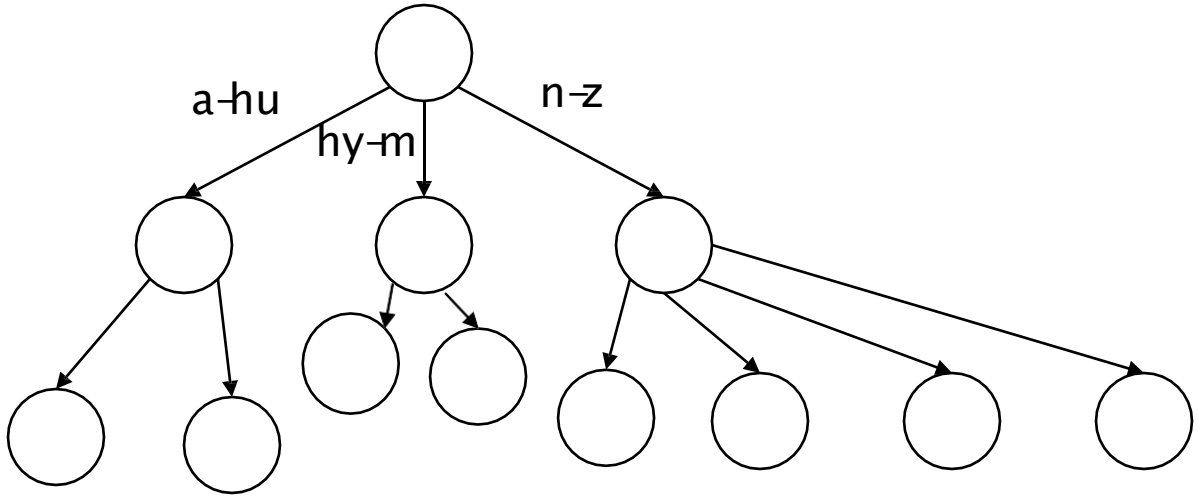
Binary tree



Binary Tree

- Cost of operations depends on height of tree.
- Keep height minimum / keep binary tree balanced: for each node, heights of subtrees differ by no more than 1. O(log M) search for balanced trees, where M is the size of the vocabulary.
- Search is slightly slower than in hashes But: re-balancing binary trees is expensive (insertion and deletion of terms).

Tree: B-tree



- Definition: Every internal nodel has a number of children in the interval [*a*,*b*] where *a*, *b* are appropriate natural numbers, e.g., [2,4].

Trees

- Simplest: binary tree
- More usual: B-trees
- Trees require a standard ordering of characters and hence strings … but we typically have one
- Pros:
  - Solves the prefix problem (terms starting with *hyp*)
- Cons:
  - Slower: O(log *M*) [and this requires *balanced* tree]
  - Rebalancing binary trees is expensive
    - But B-trees mitigate the rebalancing problem

Wild-card queries

Why do we need wild-card queries

- the user is uncertain of the spelling of a query term. (e.g., Sydney vs. Sidney).
  - This leads to a wild-card query S*dney
- the user is aware of multiple variants of spelling a term. (e.g., color vs. colour).
- the user seeks documents containing variants of a term that would be caught by stemming, (e.g., judicial vs. judiciary, leading to the wildcard query judicia*)
- the user is uncertain of the correct rendition of a foreign word or phrase (e.g., the query Universit* Stuttgart

Wild-card queries: *

- ***mon*:** find all docs containing any word beginning with “mon”.
- Easy with binary tree (or B-tree) lexicon: retrieve all words in range: ***mon*** ≤ ***w*** < ***moo***
- ****mon*:** find words ending in “mon”: harder
  - Maintain an additional B-tree for terms *backwards (reverse B-tree)*.

  Then retrieve all terms t in subtree rooted at n-o-m

Exercise: from this, how can we enumerate all terms meeting the wild-card query ***pro*cent*** ?

Query processing

- At this point, we have a list of all terms in the dictionary that match the wild-card query.
- We still have to look up the postings for each enumerated term.
- E.g., consider the query:

  se*ate** AND **fil*er

  This may result in the execution of many Boolean *AND* queries.`

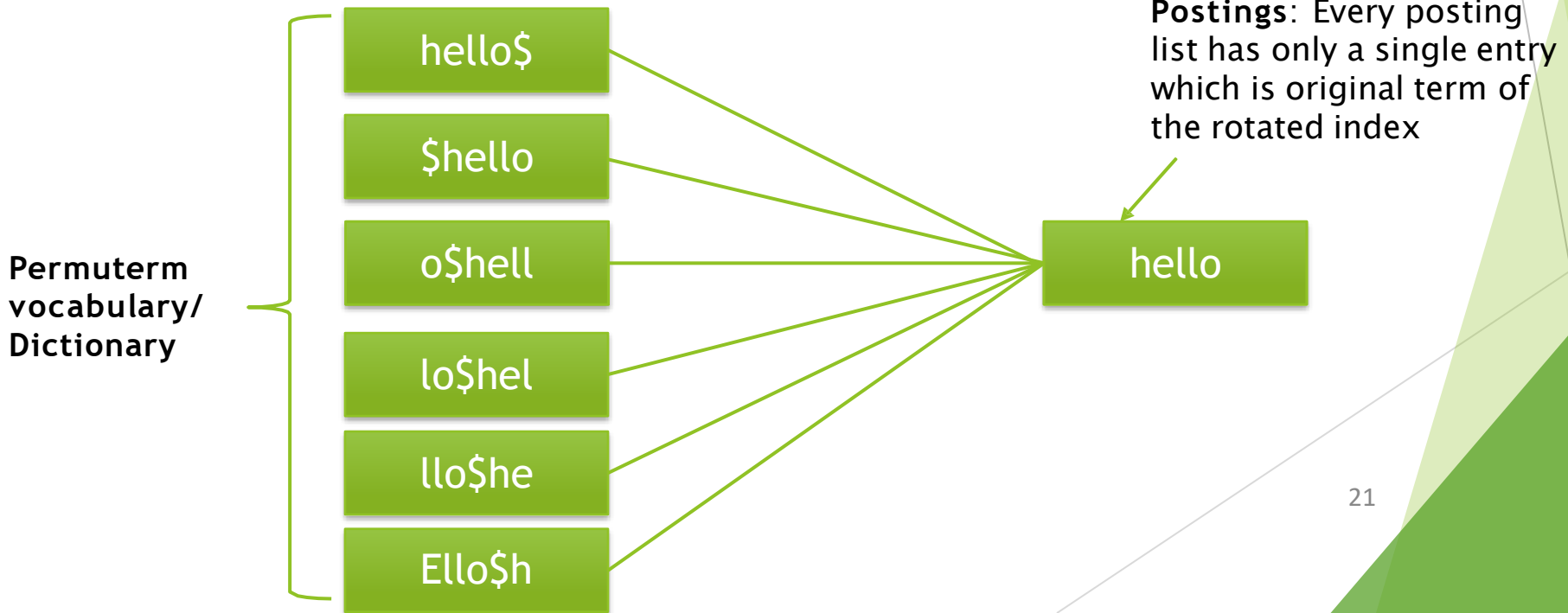
B-trees handle *'s at the end of a query term

- How can we handle *'s in the middle of query term?
  - ***co*tion***
- We could look up ***co**** AND ****tion*** in a B-tree and intersect the two term sets
  - Expensive
- The solution: transform wild-card queries so that the *'s occur at the end
- This gives rise to the **Permuterm** Index.

Permuterm index

- For term ***hello***, index under:
  - ***hello$, ello$h, llohe, lohel, o$hell, $hello***

  where $ is a special symbol.
- Permuterm Dictionary/Vocabulary



X=>String of 1 or more characters

- Queries:
  - X lookup on **X$** **X*** lookup on $X*
  - ***X** lookup on **X$*** ***X*** lookup on **X***
  - **X*Y** lookup on **Y$X*** **fi*mo*er**???
  - er$fi* -> fishmonger & filibuster
- **Y$X*???**
  - Look up for **y$x** prefix in dictionary of the permuterm index and get all the corresponding terms in the posting list for all those and collect them together.
  - Then by taking all those original terms and check them in standard inverted index to obtain Doc ids of those terms and get union of them.

Query = ***hel*o***
X=*hel*, Y=*o*
Lookup ***o$hel****

Permuterm query processing

- Rotate query wild-card to the right
- Now use B-tree lookup as before.
- *Permuterm problem: ≈ quadruples lexicon size*

Empirical observation for English.

k-gram indexes

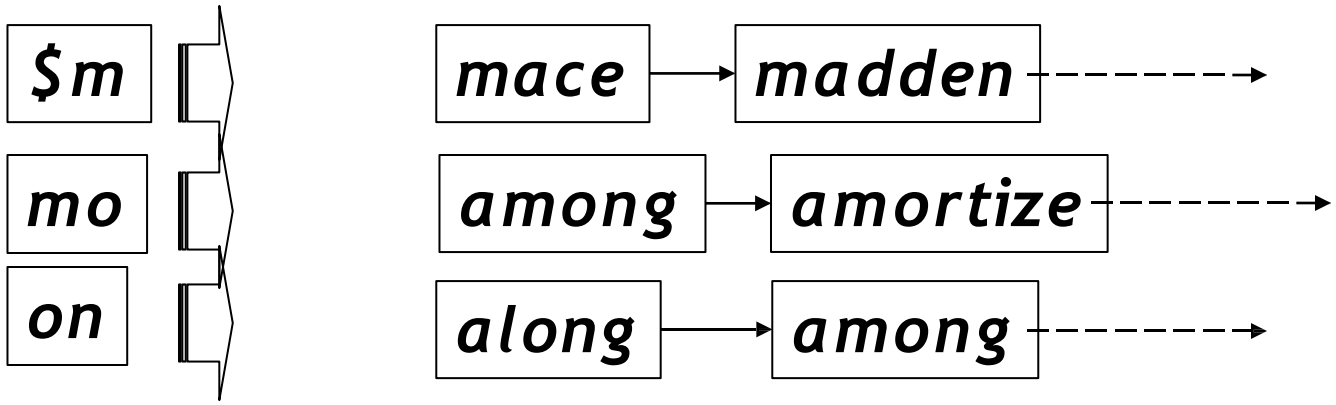
- Enumerate all *k*-grams (sequence of *k* chars) occurring in any term
- *e.g.*, from text "***April is the cruelest month***" we get the 2-grams (*bigrams*)

  a,ap,pr,ri,il,l,i,is,s,t,th,he,e,$c,cr,ru,
  ue,el,le,es,st,t$, m,mo,on,nt,h

  - $ is a special word boundary symbol
- Maintain a *second* inverted index *from bigrams to dictionary terms* that match each bigram.

Bigram index example

- The *k*-gram index finds *terms* based on a query consisting of *k*-grams (here *k*=2).

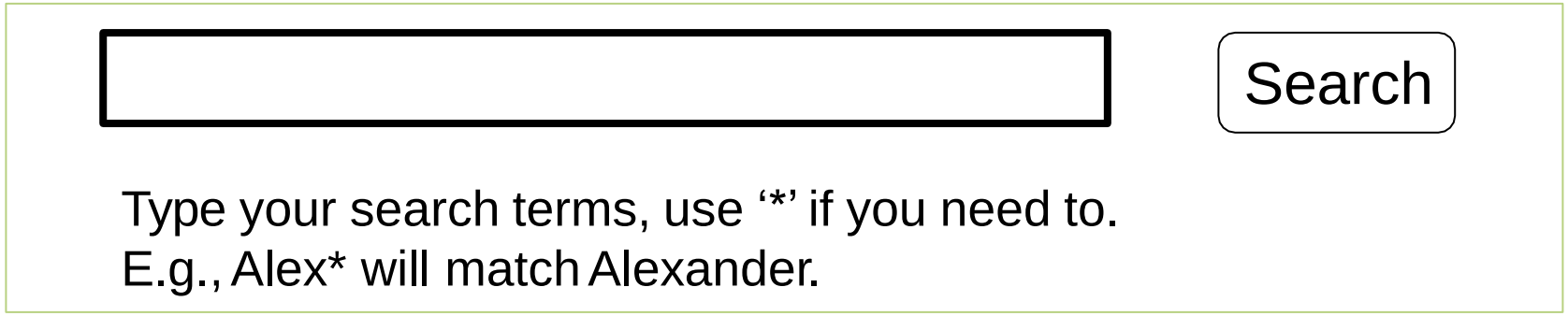


Processing wild-cards

- Query ***mon**** can now be run as
  - ***$m** AND **mo** AND **on***
- Gets terms that match AND version of our wildcard query.
- But we'd enumerate ***moon***. ← False positive
- Must post-filter these terms against query.
- Surviving enumerated terms are then looked up in the term-document inverted index.
- Fast, space efficient (compared to permuterm). (Since we are not storing every possible rotations of a single term)
- k-gram vs. permuterm index
  - k-gram index is more space-efficient
  - permuterm index does not require postfiltering always (only need for some cases with multiple wild cards).

Processing wild-card queries

- As before, we must execute a Boolean query for each enumerated, filtered term.
- Wild-cards can result in expensive query execution (very large disjunctions…)
  - pyth* AND prog*
- If you encourage "laziness" people will respond!



- Which web search engines allow wildcard queries?

Spell correction

- Two principal uses
  - Correcting document(s) being indexed
  - Correcting user queries to retrieve “right” answers
- Two main methods:
  - Isolated word
    - Check each word on its own for misspelling
    - Return the “correct” word that has the smallest distance to the misspelled word.
    - Will not catch typos resulting in correctly spelled words
    - e.g., ***from*** → ***form***
  - Context-sensitive
    - Look at surrounding words,
    - e.g., ***I flew <u>form</u> Heathrow to Narita***.

Document correction

- Especially needed for OCR'ed documents
  - Correction algorithms are tuned for this: r n/m
  - Can use domain-specific knowledge
    - E.g., OCR can confuse O and D more often than it would confuse O and I (adjacent on the QWERTY keyboard, so more likely interchanged in typing).
- But also: web pages and even printed material have typos
- Goal: the dictionary contains fewer misspellings
- But often we don't change the documents and instead fix the query-document mapping

Query mis-spellings

- Our principal focus here
  - E.g., the query ***Alanis Morisett***
- We can either
  - Retrieve documents indexed by the correct spelling, OR
  - Return several suggested alternative queries with the correct spelling
    - *Did you mean … ?*

Isolated word correction

- Fundamental premise – there is a lexicon from which the correct spellings come
- Two basic choices for this
  - A standard lexicon such as
    - Webster's English Dictionary
    - An "industry-specific" lexicon – hand-maintained
  - The lexicon of the indexed corpus
    - E.g., all words on the web
    - All names, acronyms etc.
    - (Including the mis-spellings)

Isolated word correction

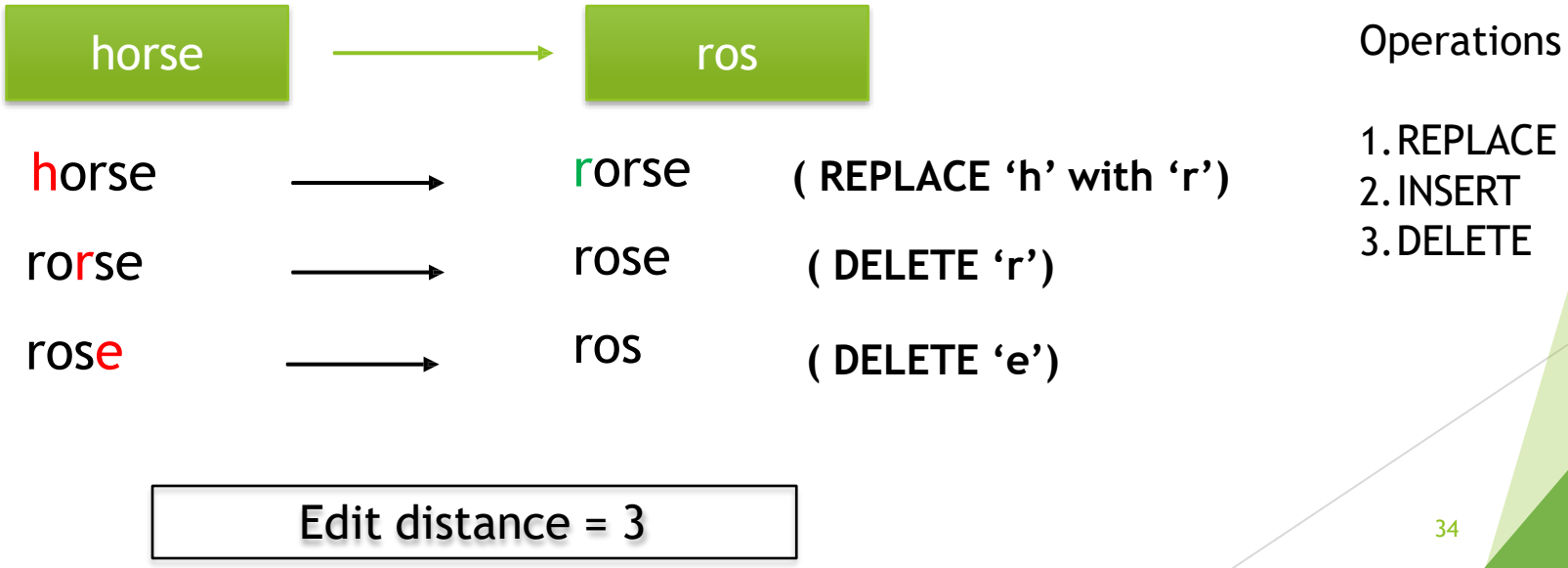
- There is a list of “correct” words – for instance a standard dictionary (Webster’s, OED. . .)
- Return the “correct” word that has the smallest distance to the misspelled word.
- Then we need a way of computing the distance between a misspelled word and a correct word
  - Edit distance (Levenshtein distance)
  - Weighted edit distance
  - *n*-gram overlap

Edit distance (Levenshtein distance)

- Given two strings S_1 and S_2, the minimum **number of operations** to convert one to the other
- Operations are typically character-level
  - Insert, Delete, Replace, (Transposition)
- E.g., the edit distance from ***dof*** to ***dog*** is 1
  - From ***cat*** to ***act*** is 2 (Just 1 with transpose.)
  - from ***cat*** to ***dog*** is 3.
- Generally found by dynamic programming.

Edit distance (Levenshtein distance)

- Edit distance between two strings s_1 and s_2 is the minimum number of basic operations that transform s_1 into s_2.
- Levenshtein distance: Acceptable operations are **insert**, **delete** and **replace**



Operations

1. REPLACE
2. INSERT
3. DELETE

Weighted edit distance

- As above, but the weight of an operation depends on the character(s) involved
  - Meant to capture OCR or keyboard errors
    Example: ***m*** more likely to be mis-typed as ***n*** than as ***q***
  - Therefore, replacing ***m*** by ***n*** is a smaller edit distance than by ***q***
  - This may be formulated as a probability model
- Requires weight matrix as input
- Modify dynamic programming to handle weights

Edit distance to all dictionary terms?

- Given a (mis-spelled) query – do we compute its edit distance to every dictionary term?
  - Expensive and slow
  - Alternative?
- How do we cut the set of candidate dictionary terms?
- One possibility is to use *n*-gram overlap for this
- This can also be used by itself for spelling correction.

n-gram overlap

- Enumerate all the *n*-grams in the query string as well as in the lexicon
- Use the *n*-gram index (recall wild-card search) to retrieve all lexicon terms matching any of the query *n*-grams
- Threshold by number of matching *n*-grams
  - Variants – weight by keyboard layout, etc.

Example with trigrams

- Suppose the text is ***november***
  - Trigrams are *nov, ove, vem, emb, mbe, ber*.
- The query is ***december***
  - Trigrams are *dec, ece, cem, emb, mbe, ber*.
- So 3 trigrams overlap (of 6 in each term)
- How can we turn this into a normalized measure of overlap?

One option – Jaccard coefficient

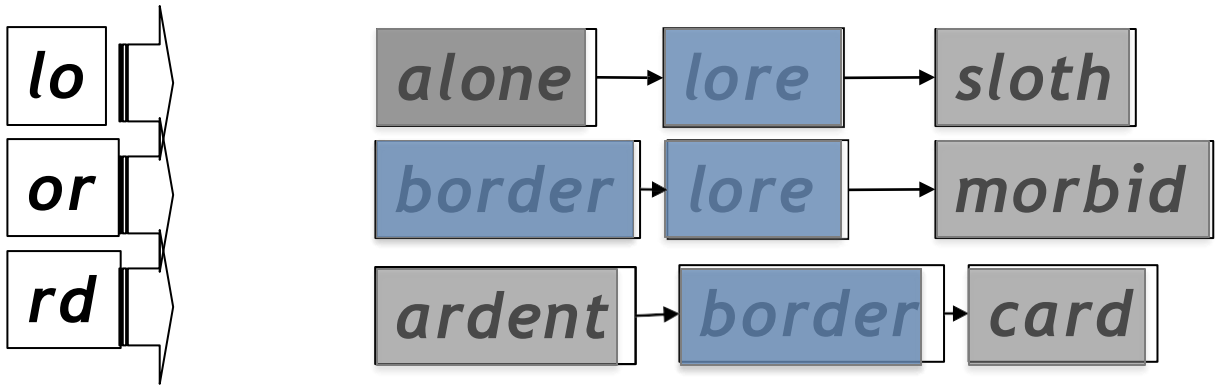
- A commonly-used measure of overlap
- Let *X* and *Y* be two sets; then the J.C. is

$$|X \cap Y| / |X \cup Y|$$

- Equals 1 when *X* and *Y* have the same elements and zero when they are disjoint
- *X* and *Y* don't have to be of the same size
- Always assigns a number between 0 and 1
  - Now threshold to decide if you have a match
  - E.g., if J.C. > 0.8, declare a match

Another option

- Consider the query ***lord*** – we wish to identify words matching 2 of its 3 bigrams (***lo***, ***or***, ***rd***)



Standard postings "merge" will enumerate …

Adapt this to using Jaccard (or another) measure.

Context-sensitive spell correction

- Text: ***I flew <u>from</u> Heathrow to Narita.***
- Consider the phrase query ***"flew <u>form</u> Heathrow"***
- We'd like to respond

  Did you mean "***flew from Heathrow***"?

because no docs matched the query phrase.

Context-sensitive correction

- Need surrounding context to catch this.
- First idea: retrieve dictionary terms close (in weighted edit distance) to each query term
- Now try all possible resulting phrases with one word “fixed” at a time
  - ***flew from heathrow***
  - ***fled form heathrow***
  - ***flea form heathrow***
- **Hit-based spelling correction:** Suggest the alternative that has lots of hits.

Exercise

- Suppose that for ***"flew form Heathrow"*** we have 7 alternatives for flew, 19 for form and 3 for heathrow.

How many "corrected" phrases will we enumerate in this scheme?

Another approach

- Biword Indexing we discussed in the Lecture 02

Soundex

Soundex

- Class of heuristics to expand a query into phonetic equivalents
  - Language specific – mainly for names
  - E.g., ***chebyshev*** → ***tchebycheff***
- Invented for the U.S. census … in 1918

Soundex – typical algorithm

- Turn every token to be indexed into a 4-character reduced form
- Do the same with query terms
- Build and search an index on the reduced forms
  - (when the query calls for a soundex match)

Soundex – typical algorithm

1. Retain the first letter of the word.
2. Change all occurrences of the following letters to '0' (zero):
   'A', E', 'I', 'O', 'U', 'H', 'W', 'Y'.
3. Change letters to digits as follows:

- B, F, P, V $\rightarrow$ 1
- C, G, J, K, Q, S, X, Z $\rightarrow$ 2
- D,T $\rightarrow$ 3
- L $\rightarrow$ 4
- M, N $\rightarrow$ 5
- R $\rightarrow$ 6

Soundex continued

4. Remove all pairs of consecutive digits.
5. Remove all zeros from the resulting string.
6. Pad the resulting string with trailing zeros and return the first four positions, which will be of the form <uppercase letter> <digit> <digit> <digit>.

E.g., ***Herman*** becomes H655.

Will ***hermann*** generate the same code?

Soundex

- Soundex is the classic algorithm, provided by most databases (Oracle, Microsoft, …)
- How useful is soundex?
- Not very – for information retrieval
- Okay for “high recall” tasks (e.g., Interpol), though biased to names of certain nationalities
- Zobel and Dart (1996) show that other algorithms for phonetic matching perform much better in the context of IR

What queries can we process?

- We have
  - Positional inverted index with skip pointers
  - Wild-card index
  - Spell-correction
  - Soundex
- Queries such as

(SPELL(moriset) /3 toron*to) OR** SOUNDEX(**chaikofski)

Exercise

- Draw yourself a diagram showing the various indexes in a search engine incorporating all the functionality we have talked about
- Identify some of the key design choices in the index pipeline:
  - Does stemming happen before the Soundex index?
  - What about *n*-grams?
- Given a query, how would you parse and dispatch sub-queries to the various indexes?

Resources

- IIR 3, MG 4.2
- Efficient spell retrieval:
  - K. Kukich. Techniques for automatically correcting words in text. ACM Computing Surveys 24(4), Dec 1992.
  - J. Zobel and P. Dart. Finding approximate matches in large lexicons. Software - practice and experience 25(3), March 1995. http://citeseer.ist.psu.edu/zobel95finding.html
  - Mikael Tillenius: Efficient Generation and Ranking of Spelling Error Corrections. Master's thesis at Sweden's Royal Institute of Technology. http://citeseer.ist.psu.edu/179155.html
- **Nice, easy reading on spell correction:**
  - Peter Norvig: How to write a spelling corrector

  http://norvig.com/spell-correct.html